

To Analyze process abstraction

Execute fork()

Understand exec() family

Analyze parent-child relationships

Inspect process tree

Practice system call tracing

Code:

```
GNU nano 8.7.1 process_lab.c *
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {

    printf("Main process PID: %d\n", getpid());

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {

        printf("Child process\n");
        printf("Child PID: %d\n", getpid());
        printf("Parent PID: %d\n", getppid());

        printf("Child executing ps...\n");

        execlp("ps", "ps", "-f", NULL);

        perror("exec");
        return 1;
    }
}
```

```

else {

    printf("Parent process\n");
    printf("Parent PID: %d\n", getpid());
    printf("Child PID: %d\n", pid);

    wait(NULL);

    printf("Child completed.\n");
}

return 0;
}
|

```

Output:

```

priyanka@DESKTOP-HOIS07A:~/OS_Skill/Week-1/process_lab$ gcc proc
ess_lab.c -o process_lab
priyanka@DESKTOP-HOIS07A:~/OS_Skill/Week-1/process_lab$ ./proces
s_lab
Main process PID: 1549
Parent process
Parent PID: 1549
Child PID: 1550
Child process
Child PID: 1550
Parent PID: 1549
Child executing ps...
UID          PID      PPID  C  STIME TTY          TIME CMD
priyanka      338        336  0  02:56 pts/0        00:00:00 -bash
priyanka     1549        338  0  03:47 pts/0        00:00:00 ./process_la
priyanka     1550       1549  0  03:47 pts/0        00:00:00 ps -f
Child completed.

```